Initial Setup Report

COMP 8967 | Fall2026 | University of Windsor
Team VitaLens (Group 1) | Business partner: PranaChain | AI Health Intelligence Engine

1. Project Details

Challenge description. A person's clinical history (labs, diagnoses, medications, notes) and their daily vitals and wearable data live in separate places, and a single reading rarely means much on its own. PranaChain wants a proof of concept that links both layers over time, explains meaningful multi-signal changes (for example, a borderline HbA1c followed by rising weight and poorer sleep) with supporting evidence, and points out missing data, without making a diagnosis. We will validate it on metabolic, cardiovascular and PCOS scenarios using 1,000 synthetic 12-month patient workbooks.

Architecture and design. We will build one hybrid pipeline: statistical methods detect change against each patient's own baseline, rules link the change to clinical history, and an LLM only explains the evidence it is given (Figure 1).

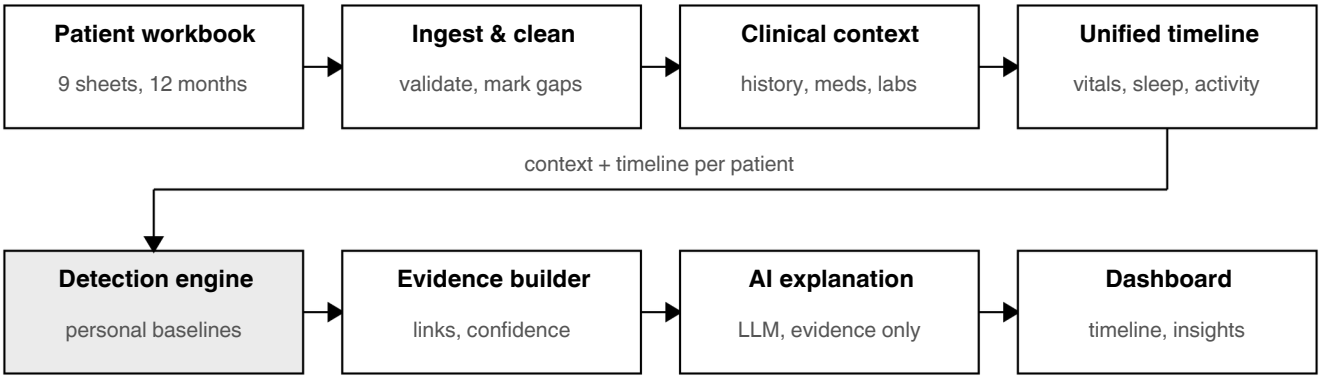


Figure 1. Solution workflow: each workbook becomes clinical context plus one timeline; changes are detected, backed with evidence,explained,and shown on a dashboard.

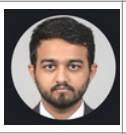
Trade-offs. Statistics + LLM instead of a pure LLM (risk of invented correlations) or deep learning (no labels, black box); personal baselines instead of fixed thresholds (finds drift, not just high/low alerts); YAML scenario patterns instead of three disease models, so new health areas need no new code. This matches the brief's five steps: understand, connect, detect, explain, ask.

Implementation strategy. Weeks 1–2: design and data audit. Weeks 3–5: data model, timeline and detection engine. Weeks 6–8: evidence builder, LLM explanations, dashboard and all three scenarios. Weeks 9–10: validation, documentation and final demo. Every weekly sprint ends with a short demo.

Technologies chosen. Python (pandas, openpyxl) to read and clean the Excel data; statsmodels, ruptures and scikit-learn for explainable change detection; Claude or GPT API with Pydantic-validated output; PostgreSQL; FastAPI; React + Recharts for the dashboard; Docker, GitHub Actions and pytest. Not chosen: LSTMs (no labels, hard to explain), MongoDB (data is tabular), Streamlit (too limited for a patient/clinician UI).

2. Team Details

Team name: VitaLens

Photo	Name	Role	Short bio	GitHub
	Uday Rasikbhai Patel	Team Lead; ML Lead (detection engine)	MAC student; data analysis, ML, time series.	UdayPatel0
	Kamran Ahmed	Backend, Data & API	MAC student; Python, REST APIs, SQL.	kamranahmed288390
	Rishi Gaurangkumar Patel	AI Lead (LLM explanations, validation)	MAC student; LLM APIs, prompt design, testing.	Rishihi29
	Nancy Sanjaykumar Parekh	Frontend & UX (dashboard)	MAC student; React, JavaScript, UI design.	Nancyparekh2502

Contribution plan. Every member writes code: Uday builds the detection engine and coordinates sprints; Kamran builds data ingestion, the database, API and CI; Rishi builds the evidence builder, LLM explanations and tests; Nancy builds the React dashboard. Each member also backs up one other area, and all code is merged through reviewed pull requests.

3. Team Agreements

Primary contact: Uday Rasikbhai Patel (Team Lead). **Communication tool:** MS Teams group chat with our GA included; two meetings per week. **Project management tool:** GitHub Projects board linked to issues and pull requests.

Contingency planning:

- **A member drops the course:** notify the GA within 24 hours; the backup owner takes over their work; adjust scope with the GA if needed.
- **Missed meetings or deadlines:** private check-in by the lead, then a written catch-up plan, then the GA is notified, then escalation to the instructor; tasks are redistributed.
- **Academic dishonesty:** we will not submit or cover for copied work; concerns are reported to the GA and instructor; AI tools and external code are cited.

4. Kickoff Meeting Summary

Business contact: Abhishek Chakrala, Founder & CEO, PranaChain (abhishek@pranachain.com), a blockchain-based, patient-owned health record platform.

Q&A: Our group did not ask questions during the meeting; the presentation and challenge brief answered our initial doubts about scope, data and deliverables.

Reflection: Before the meeting we expected to build a disease-risk prediction model with a single score. The meeting made clear that PranaChain does not want scores, threshold alerts or separate classifiers, but explainable links between past clinical findings and new daily data. This moved us to a hybrid, explainable pipeline with no-diagnosis safeguards.

5. Sprint Plan (one sprint = one week)

User stories

1. As a patient, I want to see my health timeline in one place so that I understand how my readings change over months.
2. As a patient, I want plain-language explanations of detected changes so that I know why they may matter, without a diagnosis.

3.As a clinician, I want each insight linked to past findings with its evidence so that I can judge it quickly.

4.As a developer, I want new health areas added through configuration so that the framework can grow.

Task breakdown (first 4 sprints)

Sprint	Tasks (owner)
1	Set up GitHub repo and project board (Kamran); audit the 9 data sheets (Uday, Rishi); submit setup report (All)
2	Configure CI/CD and draft database schema (Kamran); write scenario pattern specs (Uday, Rishi); build simple mock UI (Nancy)
3	Data loader and unified timeline (Kamran); baselines and trends (Uday); API endpoints (Rishi); React timeline view (Nancy)
4	Change detection and metabolic pattern (Uday); first LLM explanation (Rishi); full dataset load (Kamran); insight card (Nancy)

6. Other Details

Risks: no ground-truth labels (we will hand-build test patients per scenario); LLM hallucination or diagnostic tone (evidence-only input, schema checks); messy data (Sprint 1 audit); slow business feedback (weekly email, documented assumptions).

Assumptions: the synthetic dataset may be used in our repository and demos; a standalone batch prototype (no live PranaChain or blockchain integration) is acceptable.

Ethical considerations: insights are never presented as diagnoses and always show their data, historical link, confidence and limits; sparse data lowers confidence instead of guessing; data stays in our environment and secrets stay out of Git; patterns are checked across age and sex groups.